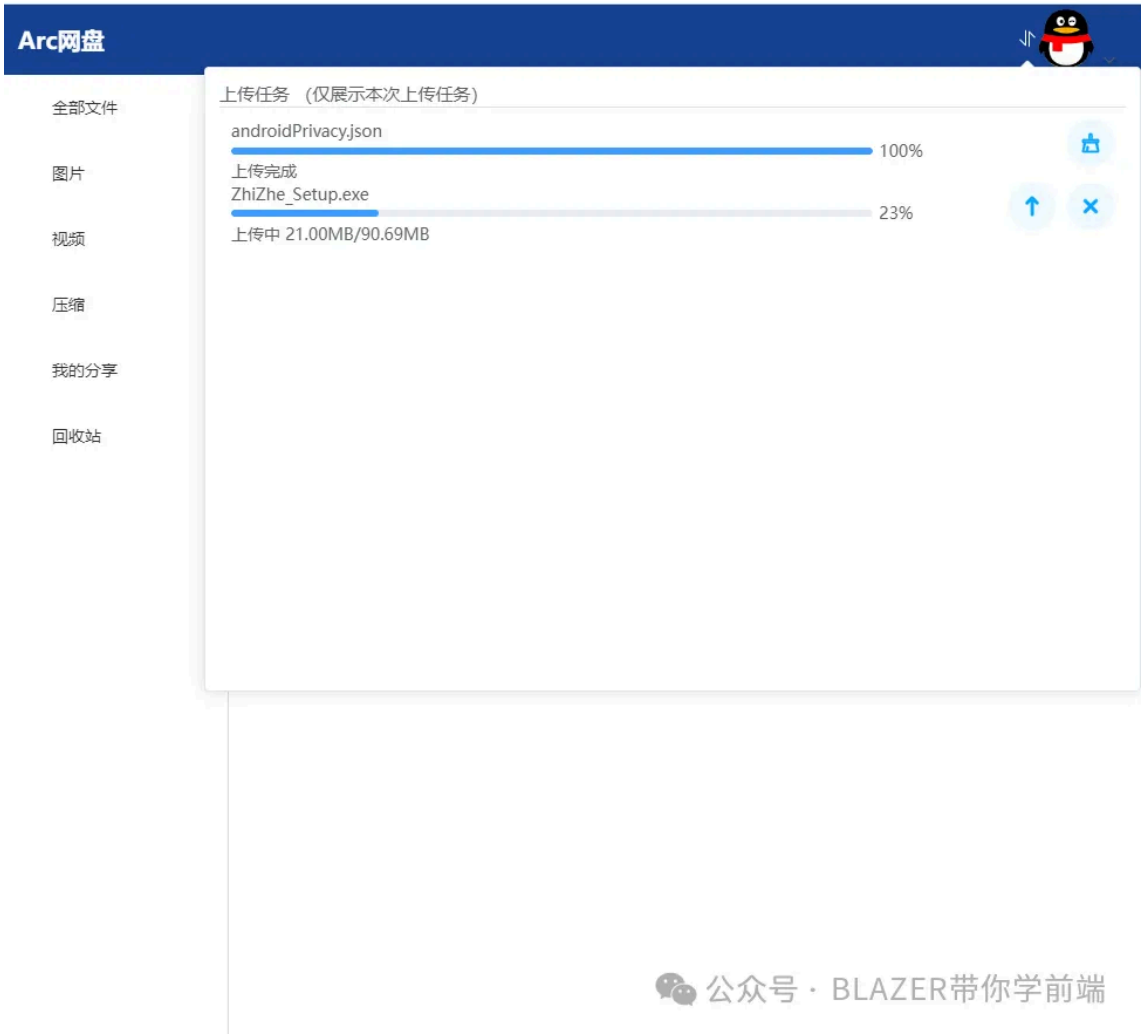
springboot+vue分片上传，秒传

Omega BLAZER带你学前端 2025年09月30日 14:12 河北

基于springboot+vue的分片上传，秒传。



1.概念

秒传：通俗的说就是把文件上传到服务器时候，通过md5进行验证，如果该文件的md5存在于数据库，那么停止上传，数据库中增加一条相同的数据，只不过是file_path变了。

分片上传：通俗的说就是将大文件传到服务器的时候分成了好几个大小相等的分片，逐批上传服务器后合并成一个

废话不说上代码

建表：

栏位 | 索引 | 外键 | 触发器 | 选项 | 注释 | SQL 预览

名	类型	长度	小数点	不是 null	
file_id	varchar	10	0	☑	1
user_id	varchar	10	0	☑	2
file_md5	varchar	255	0	☐	
file_pid	varchar	255	0	☐	
file_size	bigint	20	0	☐	
file_name	varchar	255	0	☐	
file_cover	varchar	255	0	☐	
file_path	varchar	255	0	☐	
create_time	datetime	0	0	☐	
last_update_time	datetime	0	0	☐	
folder_type	tinyint	1	0	☐	
file_category	tinyint	1	0	☐	
file_type	tinyint	1	0	☐	
status	tinyint	1	0	☐	
recovery_time	datetime	0	0	☐	
del_flag	tinyint	1	0	☐	


```
CREATE TABLE `file_info` (
  `file_id` varchar(10) NOT NULL COMMENT '文件id',
  `user_id` varchar(10) NOT NULL COMMENT '用户id',
  `file_md5` varchar(255) DEFAULT NULL COMMENT 'md5',
  `file_pid` varchar(255) DEFAULT NULL COMMENT '父级id',
  `file_size` bigint(20) DEFAULT NULL COMMENT '文件大小',
  `file_name` varchar(255) DEFAULT NULL COMMENT '文件名',
  `file_cover` varchar(255) DEFAULT NULL COMMENT '封面',
  `file_path` varchar(255) DEFAULT NULL COMMENT '文件路径',
  `create_time` datetime DEFAULT NULL COMMENT '创建时间',
  `last_update_time` datetime DEFAULT NULL COMMENT '最后修改时间',
  `folder_type` tinyint(1) DEFAULT NULL COMMENT '0文件 1目录',
  `file_category` tinyint(1) DEFAULT NULL COMMENT '文件分类1视频2音频3图片4文档5其他',
  `file_type` tinyint(1) DEFAULT NULL COMMENT '1:视频2:音频3:图片4:pdf5：doc 6：excel 7：  text8:cod
  `status` tinyint(1) DEFAULT NULL COMMENT '0转码中1转码失败2转码成功',
  `recovery_time` datetime DEFAULT NULL COMMENT '进入回收站时间',
  `del_flag` tinyint(1) DEFAULT NULL COMMENT '标记删除 0删除1回收站2正常',
  PRIMARY KEY (`file_id`,`user_id`),
  KEY `idx_create_time` (`create_time`) USING BTREE,
  KEY `idx_user_id` (`user_id`) USING BTREE,
  KEY `idx_file_pid` (`file_id`) USING BTREE,
  KEY `idx_md5` (`file_md5`) USING BTREE,
  KEY `idx_del_flag` (`del_flag`) USING BTREE,
  KEY `idx_recover_time` (`recovery_time`) USING BTREE,
  KEY `idx_status` (`status`) USING BTREE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='文件信息';
```

上传接口：必须传filePid(父级id),fileMd5(md5),chunkIndex(索引) chunks(分片数量)

```
@RequestMapping("/uploadFile")
    @GlobalInterceptor
    public Result uploadFile(HttpSession httpSession, String fileId, MultipartFile file, @VerifyPa
                             @VerifyParams(required = true) String filePid,
                             @VerifyParams(required = true) String fileMd5,
                             @VerifyParams(required = true) Integer chunkIndex,
                             @VerifyParams(required = true) Integer chunks
    ) {
        SessionWebUserDto sessionWebUserDto = (SessionWebUserDto) httpSession.getAttribute(BaseCon
        UploadResultDto uploadResultDto = fileInfoService.uploadFile(sessionWebUserDto, fileId, fi
        return Result.success(uploadResultDto);
    }
```

service

1.秒传

在第一个分片的时候对MD5进行查询，如果查询到了就是秒传，数据库添加一条新的数据，fileName=fileName+随机数+后缀名， filePath不变，更新用户使用空间

2.分片上传

1.通过分片大小file.getSize+用户使用空间+file总大小与totalSpace判断，如果大停止上传

2.创建磁盘，分片分别传入临时目录。

3.最后一个分片上传完成后记录数据库

```
    @Override
    @Transactional
    public UploadResultDto uploadFile(SessionWebUserDto sessionWebUserDto, String fileId, Multipar
        UploadResultDto uploadResultDto = new UploadResultDto();
        Boolean uploadsuccess = true;
        File templeFileFolder = null;
        //暂存临时目录
        String tempFolderName = appConfig.getProjectFolder() + BaseConstant.TEMP_FOLDER_FILE;
        try {

            //定义fileId
            if (StringTools.isEmpty(fileId)) {
                fileId = StringTools.getRandomNumber(BaseConstant.LENGTH_10);

            }

            uploadResultDto.setFileId(fileId);
            //查询当前时间
```

```
Date curdate = new Date();
//查询缓存里面的space
UserSpaceDto userSpaceDto = redisComponent.getUserSpace(sessionWebUserDto.getUserId()

//第一个分片
if (chunkIndex == 0) {
    System.out.println(chunkIndex + "chunkindex");
    //第一个分片上传时候查询mds,判断数据库是否有这个文件
    FileInfoQuery fileInfoQuery = new FileInfoQuery();
    fileInfoQuery.setFileMd5(fileMd5);
    fileInfoQuery.setStatus(2);

    //只查询第一个
    PageHelper.startPage(1, 1);
    Page<FileInfo> page = fileInfoMapper.selectByQuery(fileInfoQuery);
    List<FileInfo> list = page.getResult();

    //md5值在数据库查到了，秒传
    if (!list.isEmpty()) {
        System.out.println("12333");
        FileInfo dfileInfo = list.get(0);
        //判断文件大小,上传的文件大小+已使用的文件大小大于总大小
        if (dfileInfo.getFileSize() + userSpaceDto.getUseSpace() > userSpaceDto.getTot
            throw new BaseException("网盘空间不足，请扩容");
        }
        System.out.println("1233334");
        //容量足的话在数据库里把原来的文件copy一份给用户加上
        dfileInfo.setFileId(fileId);
        dfileInfo.setFilePid(filePid);
        dfileInfo.setUserId(sessionWebUserDto.getUserId());
        dfileInfo.setCreateTime(curdate);
        dfileInfo.setLastUpdateTime(curdate);
        dfileInfo.setStatus(2);
        dfileInfo.setDelFlag(2);
        dfileInfo.setFileMd5(fileMd5);
        System.out.println("1233335");
        //文件重命名
        fileName = autoRename(filePid, sessionWebUserDto.getUserId(), fileName);
        dfileInfo.setFileName(fileName);
        System.out.println("1233335");
        fileInfoMapper.insert(dfileInfo);
        uploadResultDto.setStatus("upload_seconds");
        //更新用户使用空间
        updateUserSpace(sessionWebUserDto, dfileInfo.getFileSize());


        return uploadResultDto;
    }
```

```
}
//判断磁盘空间是否足够
Long currentTempSize = redisComponent.getFileTemplate(sessionWebUserDto.getUserId(), f
if (file.getSize() + currentTempSize + userSpaceDto.getUseSpace() > userSpaceDto.getTo
    throw new BaseException("磁盘空间不足");
}

String currentUserFolderName = sessionWebUserDto.getUserId() + fileId;
templeFileFolder = new File(tempFolderName + currentUserFolderName);
if (!templeFileFolder.exists()) {
    System.out.println("创建磁盘");
    templeFileFolder.mkdirs();
    System.out.println("创建磁盘成功");
}
File newFile = new File(templeFileFolder.getPath() + "/" + chunkIndex);
file.transferTo(newFile);
System.out.println(chunkIndex+"chunks-1"+(chunks-1));

if (chunkIndex < chunks - 1) {

    uploadResultDto.setStatus("uploading");
    System.out.println("返回");
    redisComponent.saveFileTempSize(sessionWebUserDto.getUserId(), fileId, file.getSiz
    System.out.println("返回uploading");
    return uploadResultDto;
}

//最后一个分片上传完成，记录数据库，异步合并分片
String month = DateUtil.format(new Date(), "yyyy-MM");
System.out.println("filemonth1");
String fileSuffix = StringTools.getFileNameSuffix(fileName);
System.out.println("filemonth2");

//真实文件名

String fileRealName = currentUserFolderName + fileSuffix;
System.out.println("fileRealName"+fileRealName);

FileTypeEnums fileTypeEnums = FileTypeEnums.getFileTypeBySuffix(fileSuffix);
System.out.println("filemonth4");

//自动重命名
fileName = autoRename(filePid, sessionWebUserDto.getUserId(), fileName);
System.out.println("filemonth");
FileInfo fileInfo = new FileInfo();
fileInfo.setFileId(fileId);
fileInfo.setUserId(sessionWebUserDto.getUserId());
fileInfo.setFileMd5(fileMd5);
```

```
            fileInfo.setFileName(fileName);
            fileInfo.setFilePath(month + "/" + fileRealName);
            fileInfo.setFilePid(filePid);
            fileInfo.setCreateTime(curdate);
            fileInfo.setLastUpdateTime(curdate);
            fileInfo.setFileCategory(fileTypeEnums.getCategory().getCategory());
            fileInfo.setFileType(fileTypeEnums.getType());
            fileInfo.setStatus(0);
            fileInfo.setFolderType(0);
            fileInfo.setDelFlag(2);
            fileInfoMapper.insert(fileInfo);
            Long totalSize = redisComponent.getFileTemplate(sessionWebUserDto.getUserId(), fileId)
            System.out.println(totalSize+"totalSize");
            updateUserSpace(sessionWebUserDto, totalSize);
            uploadResultDto.setStatus("upload_finish");
            TransactionSynchronizationManager.registerSynchronization(new TransactionSynchronizati
                @Override
                public void afterCommit() {
                    System.out.println("aftercommit"+fileInfo.getFileId());
                    List<FileInfo> fileInfos1=fileInfoMapper.getBylist();
                    System.out.println(fileInfos1+"fileInfos");
                    fileInfoService.transfer(fileInfo.getFileId(), sessionWebUserDto);
                }
            });


            return uploadResultDto;


        } catch (Exception e) {
            System.out.println(e);
            uploadsuccess = false;
        } finally {
            if (!uploadsuccess && templeFileFolder != null) {
                try {
                    FileUtils.deleteDirectory(templeFileFolder);

                } catch (Exception e) {
                    throw new BaseException("删除临时失败");

                }
            }

        }
        return uploadResultDto;

    }
```

transfer

```
@Async
    public void transfer(String fileId, SessionWebUserDto sessionWebUserDto) {
        System.out.println("fileId"+fileId);
        Boolean transferSuccess = true;
        String targetFilePath = null;
        String cover = null;
        FileTypeEnums fileTypeEnums = null;
        FileInfoQuery fileInfoQuery = new FileInfoQuery();
        fileInfoQuery.setFileId(fileId);
        fileInfoQuery.setUserId(sessionWebUserDto.getUserId());
        System.out.println("fileId"+fileInfoQuery);
        System.out.println("getUserId"+sessionWebUserDto.getUserId());

        FileInfo fileInfo = fileInfoMapper.selectByQuery1(fileInfoQuery);

        try {

            if (fileInfo == null || !fileInfo.getStatus().equals(0)) {
                return;
            }
         //   临时目录
            String tempFolderName = appConfig.getProjectFolder() + BaseConstant.TEMP_FOLDER_FILE;
            String currentUserFolderName = sessionWebUserDto.getUserId() + fileId;

            File filefolder = new File(tempFolderName + currentUserFolderName);
            System.out.println("filefolder"+filefolder);
            String fileSuffix = StringTools.getFileNameSuffix(fileInfo.getFileName());
            String month = DateUtil.format(fileInfo.getCreateTime(), "yyyy-MM");
            //目标目录
            String targetFolderName = appConfig.getProjectFolder() + BaseConstant.FILE_FOLDER_FILE
            File targetFolder = new File(targetFolderName + "/" + month);
            if (!targetFolder.exists()) {
                targetFolder.mkdirs();
            }
            //真实的文件名
            String realFileName = currentUserFolderName + fileSuffix;
            targetFilePath = targetFolder.getPath() + "/" + realFileName;
            System.out.println(targetFilePath+"targetFilePath");
            //合并文件
            union(filefolder.getPath(),targetFilePath,fileInfo.getFileName(),true);
            //视频文件切割
            fileTypeEnums=FileTypeEnums.getFileTypeBySuffix(fileSuffix);
            System.out.println(fileTypeEnums+"fileTypeEnums");
            if(FileTypeEnums.VIDEO==fileTypeEnums){
                System.out.println(fileTypeEnums+"fileTypeEnums1");
```

```
                cutFileVideo(fileId,targetFilePath);
                //视频生成缩略图
                cover=month+"/"+currentUserFolderName+BaseConstant.IMAGE_PNG_SIFFOX;
                String coverPath=targetFolderName+cover;
                System.out.println(coverPath+"coverPath");
                ScaleFilter.createCover4Video(new File(targetFilePath),BaseConstant.LENGTH_150,new

            }else if(FileTypeEnums.IMAGE == fileTypeEnums){
                //生成缩略图
                cover=month+"/"+realFileName.replace(".","_.");
                String coverPath=targetFolderName+cover;
                Boolean created=ScaleFilter.createThumbnailWidthFFmpeg(new File(targetFilePath),Ba
                if(!created){
                    FileUtils.copyFile(new File(targetFilePath),new File(coverPath));

                }



            }

        } catch (Exception e) {
            transferSuccess=false;
            System.out.println(e+"ee");
            throw new BaseException("文件转码失败");

        }finally {
            FileInfo updateInfo=new FileInfo();
            updateInfo.setFileSize(new File(targetFilePath).length());
            System.out.println(new File(targetFilePath).length()+"new File(targetFilePath).length
            updateInfo.setFileCover(cover);
            updateInfo.setFileId(fileId);
            updateInfo.setUserId(sessionWebUserDto.getUserId());
            updateInfo.setStatus(transferSuccess ? 2:1);
            fileInfoMapper.update(updateInfo,fileId);
//          Long count=   fileInfoMapper.updateFileStatuswithOlderStatus(fileId,sessionWebUserDto.g
//            System.out.println(count+"count");
        }


    }
```

union() 合并所有分片成为一个大文件

```
private void union(String dirpath, String toFilePath, String fileName, Boolean delSource) {
      System.out.println("union");
      File dir = new File(dirpath);
      if (!dir.exists()) {
          throw new BaseException("目录不存在");
      }
      //获取所有的分片文件
      File[] filelist = dir.listFiles();
      //创建合并后的目标文件
      File targetFile = new File(toFilePath);
      //设置使用RandomAccessFile类以读写模式来打开文件
      RandomAccessFile writeFile = null;
      try {
          //创建writeFile类，以读写的方式打开文件
          writeFile = new RandomAccessFile(targetFile, "rw");
          //设置缓冲区，临时存储数据，统一写入TargetFile文件中
          byte[] b = new byte[1024 * 10];
          //遍历所有的分片文件
          for (int i = 0; i < filelist.length; i++) {
              int len = -1;
              File chunkfile = new File(dirpath + "/" + i);
              RandomAccessFile readfile = null;
              try {
                  //尝试从chunkfile中读文件
                  readfile = new RandomAccessFile(chunkfile, "r");
                  //循环{把读取到的字节数赋值给{len}，只要结果不是-1，那么就一直读取文件}
                  while ((len = readfile.read(b)) != -1) {
                      //把b里面的数据写到target中
                      writeFile.write(b, 0, len);
                  }
              } catch (Exception e) {
                  throw new BaseException("合并分片失败");
              } finally {
                  readfile.close();
              }
          }

      } catch (Exception e) {
          throw new BaseException("合并分片失败");
      }
      finally {
          if(null!=writeFile){
              try {
                  writeFile.close();
              }catch (Exception e){
                  e.printStackTrace();
              }
          }
```

```
        if(delSource && dir.exists()){
            try {
                System.out.println(dir+"dirrs");
                FileUtils.deleteDirectory(dir);
            }catch (Exception e){
                e.printStackTrace();
            }
        }
    }
}
```

前端逻辑

```
let fileitem = {
    file: e.file,
    md5: null,
    md5Progress: 0,
    fileName: e.file.name,
    status: status.init.value,
    //上传进度
    chunkIndex: 0,
    progress: 0,
    uploadSize: 0,
    filePid: pid,
    uid: e.file.uid,
    pause: false,
    totalSize: e.file.size
}



filelist.value.unshift(fileitem);
if (fileitem.totalSize == 0) {
    fileitem.status = status.emptyfile
    return
}
//计算md5
let md5fildeUId = await computedMd5(fileitem);
console.log(getFileByUid(md5fildeUId).md5, "mds")
console.log(md5fildeUId, "mdtr")
if (md5fildeUId == null) {
    return
}
console.log(filelist.value)
```

```
    uploadFilled(md5fildeUId)
```

计算md5(computedMd5)

```
//计算md5
const computedMd5 = (fileitem) => {
    console.log(fileitem, "file")


    let file = fileitem.file;

    let blogSlice = File.prototype.slice || File.prototype.mozSlice || File.prototype.webkitSlice
    let chunks = Math.ceil(file.size / chunkSize);
    console.log(chunks, "chunks")
    let currentChunk = 0;
    let spark = new SparkMD5.ArrayBuffer();
    let fileReader = new FileReader();
    let loadNext = () => {
        let start = currentChunk * chunkSize;
        let end = start + chunkSize >= file.size ? file.size : start + chunkSize;
        fileReader.readAsArrayBuffer(blogSlice.call(file, start, end))

    }
    loadNext();
    return new Promise((resolve, reject) => {
        let resultFile = getFileByUid(file.uid)
        fileReader.onload = (e) => {
            spark.append(e.target.result)
            currentChunk++;
            if (currentChunk < chunks) {

                let percent = Math.floor((currentChunk / chunks) * 100)
                resultFile.md5Progress = percent;
                loadNext()
            } else {
                let md5 = spark.end();
                spark.destroy()
                resultFile.md5Progress = 100
                resultFile.status = status.uploading.value;
                resultFile.md5 = md5;
                resolve(fileitem.uid)
            }
        }
        fileReader.onerror = () => {
            resultFile.md5Progress = -1
```

```
            resultFile.status = status.fail.value
            resolve(fileitem.uid)
        }
    }).catch(e => {
        return null;
    })



}
```

上传

```
const uploadFilled = async (md5fildeUId, chunkIndex) => {

    chunkIndex = chunkIndex ? chunkIndex : 0;

    let currentFile = getFileByUid(md5fildeUId);
    console.log(currentFile, "currentFile")
    const file = currentFile.file;
    console.log(file, "filsd")

    const fileSize = currentFile.totalSize;
    const chunks = Math.ceil(fileSize / chunkSize);
    for (let i = chunkIndex; i < chunks; i++) {
        let delindex = dellist.value.indexOf(md5fildeUId);
        if (delindex != -1) {
            dellist.value.splice(delindex, 1);
            break;
        }
        currentFile = getFileByUid(md5fildeUId)
        if (currentFile.pause) {
            break
        }
        let start = i * chunkSize;
        console.log(start, "start", i)
        let end = start + chunkSize >= fileSize ? fileSize : start + chunkSize;
        console.log(end, "end", i)
        let chunkfile = file.slice(start, end);
        console.log(chunkfile, "fileuploading")
        let data = {
            file: chunkfile,
            fileName: currentFile.fileName,
            fileMd5: currentFile.md5,
            chunkIndex: i,
```

```
            chunks: chunks,
            fileId: currentFile.fileId,
            filePid: currentFile.filePid,

        }


        let updateResult = await service.post("/file/uploadFile", data)
        console.log(updateResult, "updateresult")

        currentFile.fileId = updateResult.data.fileId;
        console.log(filelist, "filese")
        if (updateResult.data.status !== null) {
            currentFile.status = status[updateResult.data.status].value

        } else {
            currentFile.status = status["fail"].value
            ElMessage({
                type: "warning",
                text: "上传失败，请扩容或联系管理员"
            })

        }

        currentFile.chunkIndex = i;
        currentFile.uploadSize = i * chunkSize;
        currentFile.progress = Math.floor((currentFile.uploadSize / fileSize) * 100)
        if (updateResult.data.status == status.upload_seconds.value || updateResult.data.status ==
            currentFile.progress = 100
            break;
        }

        console.log(updateResult)



    }
    emit("showlist")
    //分片上传
}
```

如何获取？私聊wx:ws_1112223334获取源码

最近腾讯云9月份有活动，想买腾讯云的可以找我

需要了解服务器和域名的购买以及返佣政策的可以找我，我有朋友做这方面对这些比较了解，加我微信就好

----------广而告之----------

鸿蒙和JAVA学习群已经建好，想学鸿蒙和java的小伙伴们可以加群了，目前群里已经满200人了，500人时候不再拉人~，一个萝卜一个坑哦。新开了2群，适合想学java+vue的。想了解的加v：ws_1112223334